

Python per l'Ingegneria Strutturale

Dispensa completa — Lezioni 1÷9

NumPy · Matplotlib · FEM · OpenSeesPy · NTC2018

Ing. Ramiro Klein | 2025–2026

Nozioni fondamentali di Python

Liste vs Array NumPy

In Python esistono due modi principali per raccogliere una sequenza di numeri:

- Lista → contenitore generico, non matematico
- Array NumPy → oggetto matematico, ottimizzato per il calcolo numerico

Differenza chiave:

```
lista = [1, 2, 3, 4]
array = np.array([1, 2, 3, 4])

lista * 2      # → [1,2,3,4,1,2,3,4]  (duplica gli elementi!)
array * 2      # → [2,4,6,8]          (moltiplica ogni elemento)
```

🔗 In ingegneria si usano SEMPRE gli array NumPy per i calcoli — le liste servono solo come contenitori generici (es. lista di tag di elementi).

Slicing (indicizzazione)

Python usa indici che partono da 0. La sintassi [inizio:fine] restituisce gli elementi dall'indice 'inizio' fino a 'fine-1' (fine escluso).

```
lista = [10, 20, 30, 40, 50]
lista[0]      # → 10      (primo elemento)
lista[-1]     # → 50      (ultimo elemento)
lista[1:3]    # → [20, 30] (indici 1 e 2, il 3 è escluso)
```

🔗 Regola: fine - inizio = numero di elementi restituiti. Esempio: [1:3] → 3-1 = 2 elementi.

Lo slicing si usa anche sulle matrici NumPy per l'assemblaggio FEM:

```
K[0:4, 0:4] += k1  # blocco 4x4 in alto a sinistra
K[2:6, 2:6] += k2  # blocco 4x4 sovrapposto (nodo condiviso)
```

Dizionari

I dizionari permettono di associare un nome (chiave) a un valore, invece di usare un indice numerico. Sono ideali per database di profili strutturali.

```
profili = {
    "IPE200": 220.6,
    "IPE330": 804.3,
}

profili["IPE330"]  # → 804.3 (accesso per chiave)
```

🔗 Lista → accesso per posizione (lista[0]). Dizionario → accesso per chiave (profili["IPE330"]).

Cicli for

Due usi tipici in ingegneria strutturale:

```
# Ciclo su lista di numeri
for ele in [1, 2, 3, 4, 5, 6]:
    ops.eleLoad('-ele', ele, ...)

# Ciclo su dizionario (chiave + valore insieme)
for nome, wpl in profili.items():
    print(nome, wpl)
```

Funzioni e argomenti di default

Una funzione si definisce con 'def' e restituisce risultati con 'return'. Un argomento con valore di default è opzionale quando si chiama la funzione.

```
def verifica_flessione(M_max, w_pl, f_yk, gamma_M0=1.05):  
    # gamma_M0 ha default 1.05 → non serve specificarlo ogni volta  
    f_yd = f_yk / gamma_M0  
    ...  
    return verifica  
  
verifica_flessione(187.5, 220.6, 275)          # usa gamma_M0=1.05  
verifica_flessione(187.5, 220.6, 275, 1.10)   # sovrascrive con 1.10
```

🔗 L'indentazione in Python NON è estetica — definisce la struttura del codice. Tutto il codice dentro una funzione deve essere rientrato di 4 spazi.

f-string e formattazione numeri

Le f-string permettono di inserire variabili e formati numerici direttamente dentro le stringhe di testo:

```
valore = 187.5678  
print(f"{valore:.0f}")    # → 188        (0 decimali)  
print(f"{valore:.1f}")    # → 187.6      (1 decimale)  
print(f"{valore:.2f}")    # → 187.57     (2 decimali)  
print(f"{valore:10.2f}")  # → "      187.57" (larghezza 10, allineato)  
print(f"{valore:e}")      # → 1.875678e+02 (notazione scientifica)
```

🔗 La sintassi `:.1f` si legge: ':' applica formato, '.1' una cifra decimale, 'f' numero float. Utile per allineare colonne in output tabellari.

Lezione 1 — Operazioni tra vettori con NumPy

Prima lezione pratica: creare array, calcolare somme, medie e applicare fattori di sicurezza. Concetto chiave: il broadcasting NumPy (un'operazione scalare si applica a ogni elemento dell'array).

```
import numpy as np

# Forze nodali in kN (negative = verso il basso)
F = np.array([0, -30, -25, -10, 0])

F_tot = np.sum(F)      # somma di tutti gli elementi
F_mean = np.mean(F)    # media

# Fattore di sicurezza NTC2018 – broadcasting
gamma = 1.5
F_design = F * gamma    # moltiplica OGNI elemento per 1.5

# Tensione normale  $\sigma = F/A$  (divisione elemento per elemento)
A = np.array([25, 30, 30, 25]) # cm2
A_m2 = A / 10000                # conversione in m2
sigma = F_design[1:] / A_m2      # [1:] esclude il primo elemento (zero)
```

☞ `F_design[1:]` usa lo slicing per escludere il nodo senza carico. La divisione tra due array divide gli elementi corrispondenti uno a uno.

Lezione 2 — Sistemi lineari: reazioni vincolari

Il calcolo delle reazioni vincolari di una trave appoggiata si riduce a risolvere un sistema lineare $A \cdot x = b$. NumPy lo risolve in una riga con `np.linalg.solve()`.

Schema statico: trave con luce L , carico distribuito q , carico concentrato P a distanza a da sinistra.

Equazioni di equilibrio:

- $\Sigma F_y = 0 \rightarrow R_A + R_B = q \cdot L + P$
- $\Sigma M(A) = 0 \rightarrow R_B \cdot L = q \cdot L^2/2 + P \cdot a$

```
import numpy as np

L = 10.0 # luce [m]
q = 20.0 # carico distribuito [kN/m]
P = 60.0 # carico concentrato [kN]
a = 7.0  # posizione P da sinistra [m]

# Matrice dei coefficienti A
A = np.array([[1, 1],
              [0, L]])

# Vettore termini noti b (calcolato dai dati fisici)
b = np.array([q*L + P,
              q*L*L/2 + P*a])

# Soluzione del sistema A·x = b
x = np.linalg.solve(A, b)

print(f"RA = {x[0]:.1f} kN")
print(f"RB = {x[1]:.1f} kN")
```

Costruire b dai dati fisici (non dai numeri già calcolati) permette di cambiare L , q , P , a e rieseguire senza modificare altro codice.

Differenza fondamentale rispetto alla trave FEM: qui usiamo le equazioni di equilibrio globale (statica), non la matrice di rigidezza.

Lezione 3 — Diagrammi di taglio e momento flettente

Con Matplotlib si tracciano i diagrammi di sollecitazione. `np.linspace()` crea una griglia di punti lungo la trave per valutare $V(x)$ e $M(x)$ con continuità.

```
import numpy as np
import matplotlib.pyplot as plt

L, q, P, a = 8.0, 15.0, 40.0, 3.0
RA, RB = 80.5, 79.5

x = np.linspace(0, L, 500) # 500 punti da 0 a L

# Taglio V(x): salto in corrispondenza del carico concentrato
V = RA - q*x - P*(x>=a).astype(float)

# Momento M(x): integrale di V(x)
M = RA*x - q*x**2/2 - P*np.maximum(x-a, 0)

# Valore e posizione del momento massimo
M_max = np.max(M)
x_max = x[np.argmax(M)] # np.argmax restituisce l'INDICE del massimo

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10,5))
ax1.fill_between(x, V, alpha=0.3, color='steelblue')
ax2.fill_between(x, M, alpha=0.3, color='coral')
plt.tight_layout()
plt.show()
```

☞ $V = dM/dx$: il momento è massimo dove il taglio cambia segno (si annulla). Questa relazione permette di leggere fisicamente i diagrammi.

☞ `plt.show()` è bloccante: ferma il codice finché non si chiude la finestra. Usare `plt.show(block=False)` per non bloccare.

Lezione 4 — Funzioni riutilizzabili

Raccogliere il codice in una funzione permette di analizzare qualsiasi trave cambiando solo i parametri di input, senza riscrivere la logica ogni volta.

```
def analisi_trave(L, q, P, a):
    # Reazioni vincolari
    A = np.array([[1, 1], [0, L]])
    b = np.array([q*L + P, q*L*L/2 + P*a])
    R = np.linalg.solve(A, b)
    RA, RB = R[0], R[1]    # spaccettamento in due variabili

    # Diagrammi
    x = np.linspace(0, L, 500)
    V = RA - q*x - P*(x>=a).astype(float)
    M = RA*x - q*x**2/2 - P*np.maximum(x-a, 0)

    print(f"RA = {RA:.1f} kN, RB = {RB:.1f} kN")
    print(f"Mmax = {np.max(M):.1f} kNm a x = {x[np.argmax(M)]:.2f} m")

    # ... grafico ...
    return x, V, M

# Analisi di due travi diverse con la stessa funzione
x, V, M = analisi_trave(L=8.0, q=15.0, P=40.0, a=3.0)
x, V, M = analisi_trave(L=10.0, q=20.0, P=60.0, a=7.0)
```

La notazione `L=8.0` si chiama 'argomento con keyword' — rende il codice autoesplicativo e permette di passare gli argomenti in qualsiasi ordine.

Lezione 5 — Metodo degli Elementi Finiti (FEM)

Implementazione manuale del FEM per una trave a 2 elementi. La matrice di rigidezza di un elemento trave Eulero-Bernoulli ha dimensione 4×4 (2 gdl per nodo: traslazione verticale v e rotazione θ).

Matrice di rigidezza locale

```
def k_locale(E, I, L):
    """Matrice rigidezza elem. trave Eulero-Bernoulli 4x4"""
    c = E * I / L**3
    return c * np.array([
        [ 12, 6*L, -12, 6*L ],
        [ 6*L, 4*L**2, -6*L, 2*L**2],
        [-12, -6*L, 12, -6*L ],
        [ 6*L, 2*L**2, -6*L, 4*L**2],
    ])
```

Assemblaggio matrice globale

Ogni elemento contribuisce alla matrice globale nella posizione corrispondente ai suoi gdl. Elementi adiacenti condividono il nodo centrale — i contributi si sommano.

```
K = np.zeros((6, 6))    # 3 nodi × 2 gdl = 6 gdl totali

K[0:4, 0:4] += k1       # elemento 1: gdl 0,1,2,3
K[2:6, 2:6] += k2       # elemento 2: gdl 2,3,4,5 (nodo 1 condiviso!)
```

Condizioni al contorno e soluzione

```
F = np.zeros(6)
F[2] = -50.0           # carico al nodo centrale

gdl_liberi = [1, 2, 3, 5] # gdl non vincolati
K_rid = K[np.ix_(gdl_liberi, gdl_liberi)]
F_rid = F[gdl_liberi]

u_rid = np.linalg.solve(K_rid, F_rid)

# Ricostruzione vettore completo
u = np.zeros(6)
for i, gdl in enumerate(gdl_liberi):
    u[gdl] = u_rid[i]

print(f"Freccia massima: {min(u)*1000:.2f} mm")
```

[np.ix_\(\)](#) crea un selettore per righe e colonne contemporaneamente — permette di estrarre il sottoblocco K_{rid} in modo compatto.

Lezione 6 — Verifica sezioni in acciaio (NTC2018)

Verifica di resistenza a flessione semplice secondo NTC2018/EC3. La condizione è: $\sigma = M_{Ed} / W_{pl} \leq f_{yd} = f_{yk} / \gamma_{M0}$

```
def verifica_flessione(M_max, W_pl, f_yk, gamma_M0=1.05):
    f_yd = f_yk / gamma_M0          # tensione di progetto [MPa]
    M_Nmm = M_max * 1e6             # conversione kNm → N·mm
    W_mm3 = W_pl * 1e3              # conversione cm³ → mm³
    sigma = M_Nmm / W_mm3           # tensione di calcolo [MPa]
    verifica = sigma <= f_yd
    print(f"σ = {sigma:.1f} MPa | f_yd = {f_yd:.1f} MPa")
    print(f"Verifica: {'OK' if verifica else 'NON SODDISFATTA'}")
    return verifica

# Pre-dimensionamento automatico con dizionario profili
profili = {
    "IPE200": 220.6, "IPE220": 285.4, "IPE240": 366.6,
    "IPE270": 484.0, "IPE300": 628.4, "IPE330": 804.3,
}

for nome, Wpl in profili.items():
    ok = verifica_flessione(187.5, Wpl, 275)
    if ok:
        print(f"Profilo minimo: {nome}")
        break
```

Il ciclo for con break si ferma al primo profilo che soddisfa la verifica — simula la logica di pre-dimensionamento automatico per profili tabulati.

gamma_M0 = 1.05 per acciaio S235/S275/S355 (NTC2018 §4.2.4.1.2). Modificare per acciai inossidabili o connessioni.

Lezione 7 — Primo modello OpenSeesPy

Flusso di lavoro OpenSeesPy

Ogni modello OpenSeesPy segue sempre questo ordine obbligatorio:

- 1. wipe() — pulisce la memoria da modelli precedenti
- 2. model() — definisce dimensioni e gradi di libertà
- 3. node() — posiziona i nodi
- 4. fix() — applica i vincoli
- 5. geomTransf() — trasformazione geometrica
- 6. element() — definisce gli elementi
- 7. timeSeries() + pattern() — definisce i carichi
- 8. Configurazione analisi + analyze()
- 9. Lettura risultati

Vincoli: ops.fix()

La funzione fix(nodeTag, ux, uy, rz) blocca i gradi di libertà del nodo. 1=bloccato, 0=libero.

```
ops.fix(1, 0, 1, 0) # appoggio semplice: solo uy bloccato
ops.fix(1, 1, 1, 0) # cerniera: ux e uy bloccati
ops.fix(1, 1, 1, 1) # incastro: tutti e 3 i gdl bloccati
```

Trave appoggiata con carico misto

```
import openseespy.opensees as ops

ops.wipe()
ops.model('basic', '-ndm', 2, '-ndf', 3)

ops.node(1, 0.0, 0.0) # appoggio fisso
ops.node(2, 3.0, 0.0) # punto di carico P
ops.node(3, 8.0, 0.0) # appoggio scorrevole

ops.fix(1, 1, 1, 0) # fisso: blocca ux e uy
ops.fix(3, 0, 1, 0) # scorrevole: blocca solo uy

E = 210e6; A = 28.5e-4; I = 1943e-8
ops.geomTransf('Linear', 1)
ops.element('elasticBeamColumn', 1, 1, 2, A, E, I, 1)
ops.element('elasticBeamColumn', 2, 2, 3, A, E, I, 1)

ops.timeSeries('Linear', 1)
ops.pattern('Plain', 1, 1)
ops.load(2, 0.0, -40.0, 0.0) # concentrato
ops.eleLoad('-ele', 1, '-type', '-beamUniform', -15.0) # distribuito ele 1
ops.eleLoad('-ele', 2, '-type', '-beamUniform', -15.0) # distribuito ele 2

ops.system('BandSPD')
ops.numberer('RCM')
ops.constraints('Plain')
ops.integrator('LoadControl', 1.0)
ops.algorithm('Linear')
ops.analysis('Static')
```

```
ops.analyze(1)

ops.reactions()
R1 = ops.nodeReaction(1)
print(f"RA = {R1[1]:.1f} kN")
```

Lezione 8 — Spostamenti nodali in OpenSeesPy

Dopo l'analisi, `ops.nodeDisp(tag)` restituisce il vettore degli spostamenti `[ux, uy, rz]` per ogni nodo. La freccia massima corrisponde al valore minimo (più negativo) di `uy`.

```
u1 = ops.nodeDisp(1)
u2 = ops.nodeDisp(2)
u3 = ops.nodeDisp(3)

print(f"uy nodo 2: {u2[1]*1000:.1f} mm")    # freccia al punto di carico

# Verifica SLE – limite L/250
freccia = abs(u2[1])
limite = L / 250
print(f"Freccia: {freccia*1000:.1f} mm | Limite: {limite*1000:.1f} mm")
print("SLE OK" if freccia <= limite else "SLE NON SODDISFATTA")
```

🔗 I nodi vincolati (appoggi/incastri) hanno $u_y=0$ come atteso. Le rotazioni r_z ai nodi liberi rappresentano la pendenza della linea elastica.

❑ *Verificare SEMPRE sia la resistenza (SLU) sia la deformabilità (SLE). Una sezione può passare la verifica di resistenza ma avere freccia eccessiva.*

Lezione 9 — Telaio 2D a 3 piani

Geometria e numerazione nodi

Telaio con 2 pilastri, 3 travi orizzontali (3 piani), incastri alla base. 8 nodi totali, 9 elementi (6 pilastri + 3 travi).

```
H = 3.0 # altezza interpiano [m]
L = 5.0 # luce [m]

# Colonna sinistra (x=0): nodi 1-4
ops.node(1, 0.0, 0.0) # base
ops.node(2, 0.0, H)   # piano 1
ops.node(3, 0.0, 2*H) # piano 2
ops.node(4, 0.0, 3*H) # piano 3 (sommità)

# Colonna destra (x=L): nodi 5-8
ops.node(5, L, 0.0)   # base
ops.node(6, L, H)     # piano 1
ops.node(7, L, 2*H)   # piano 2
ops.node(8, L, 3*H)   # piano 3

ops.fix(1, 1, 1, 1)   # incastro base sinistra
ops.fix(5, 1, 1, 1)   # incastro base destra
```

Elementi e trasformazioni geometriche

```
# Due trasformazioni separate: una per pilastri, una per travi
ops.geomTransf('Linear', 1) # pilastri
ops.geomTransf('Linear', 2) # travi

# Pilastri sinistra
ops.element('elasticBeamColumn', 1, 1, 2, A_col, E, I_col, 1)
ops.element('elasticBeamColumn', 2, 2, 3, A_col, E, I_col, 1)
ops.element('elasticBeamColumn', 3, 3, 4, A_col, E, I_col, 1)

# Pilastri destra
ops.element('elasticBeamColumn', 4, 5, 6, A_col, E, I_col, 1)
ops.element('elasticBeamColumn', 5, 6, 7, A_col, E, I_col, 1)
ops.element('elasticBeamColumn', 6, 7, 8, A_col, E, I_col, 1)

# Travi orizzontali
ops.element('elasticBeamColumn', 7, 2, 6, A_trave, E, I_trave, 2)
ops.element('elasticBeamColumn', 8, 3, 7, A_trave, E, I_trave, 2)
ops.element('elasticBeamColumn', 9, 4, 8, A_trave, E, I_trave, 2)
```

Carichi verticali — pattern separati

Separare i carichi in pattern distinti (G1, G2, Q) è fondamentale per applicare correttamente i coefficienti γ della combinazione NTC2018.

```
gamma_acciaio = 78.5 # kN/m³
pp_col = gamma_acciaio * A_col # peso proprio pilastro [kN/m]
pp_trave = gamma_acciaio * A_trave # peso proprio trave [kN/m]
interasse = 4.0 # distanza tra telai paralleli [m]
q_G2 = 2.0 * interasse # permanenti solaio [kN/m]
q_Q = 2.0 * interasse # variabili solaio [kN/m]

# Pattern 1: Peso proprio + permanenti (G1 + G2)
```

```
ops.timeSeries('Linear', 1)
ops.pattern('Plain', 1, 1)
for ele in [1,2,3,4,5,6]: # pilastri – carico ASSIALE
    ops.eleLoad('-ele', ele, '-type', '-beamUniform', 0.0, -pp_col)
for ele in [7,8,9]:      # travi – carico TRASVERSALE
    ops.eleLoad('-ele', ele, '-type', '-beamUniform', -(pp_trave+q_G2))

# Pattern 2: Variabili (Q)
ops.timeSeries('Linear', 2)
ops.pattern('Plain', 2, 2)
for ele in [7,8,9]:
    ops.eleLoad('-ele', ele, '-type', '-beamUniform', -q_Q)
```

□ *Per i pilastri verticali usare 0.0 come primo argomento (trasversale=0) e il peso come secondo (assiale). Viceversa per le travi orizzontali si usa solo il primo argomento.*

Carico del vento — distribuzione con l'altezza

Pressione crescente con l'altezza, convertita in forze concentrate ai piani (ipotesi di diaframma rigido):

```
import numpy as np

p_base      = 0.5    # kN/m² pressione di riferimento
incremento  = 0.15   # kN/m² per metro di altezza
altezze     = np.array([H, 2*H, 3*H]) # quote dei piani
h_influenza = np.array([H, H, H/2])   # area verticale di influenza

p_vento = p_base + incremento * altezze
F_vento = p_vento * interasse * h_influenza

# Pattern 3: Vento (forze concentrate orizzontali)
ops.timeSeries('Linear', 3)
ops.pattern('Plain', 3, 3)
ops.load(2, F_vento[0], 0.0, 0.0) # piano 1
ops.load(3, F_vento[1], 0.0, 0.0) # piano 2
ops.load(4, F_vento[2], 0.0, 0.0) # piano 3
```

🔗 *Il vento genera un momento ribaltante alla base: la reazione verticale F_y aumenta sul lato sottovento e diminuisce sul lato sopravvento. Questo effetto è fondamentale nella verifica dei pilastri.*

🔗 *L'interasse rappresenta la 'fascia di carico' del telaio: la porzione di solaio/facciata il cui carico è portato da questo telaio specifico.*

Visualizzazione del telaio

```
import matplotlib.pyplot as plt

def plot_telaio():
    fig, ax = plt.subplots(figsize=(6, 8))
    for ele in ops.getEleTags():
        nodi = ops.eleNodes(ele)
        x = [ops.nodeCoord(nodi[0])[0], ops.nodeCoord(nodi[1])[0]]
        y = [ops.nodeCoord(nodi[0])[1], ops.nodeCoord(nodi[1])[1]]
        ax.plot(x, y, 'b-', linewidth=2)
    for nodo in ops.getNodeTags():
        x, y = ops.nodeCoord(nodo)
        ax.plot(x, y, 'ko', markersize=6)
        ax.annotate(str(nodo), (x,y), xytext=(8,5),
                    textcoords='offset points')
    ax.set_aspect('equal')
    plt.show()
```


Riepilogo — Le 7 righe di analisi OpenSeesPy

Queste righe compaiono in ogni analisi statica lineare. Ecco cosa fa ciascuna:

```
ops.system('BandSPD')      # organizza K in memoria (matrice a banda simmetrica)
ops.numberer('RCM')        # riordina i gdl per minimizzare la banda di K
ops.constraints('Plain')    # gestisce i vincoli eliminando le righe/colonne
bloccate
ops.integrator('LoadControl', 1.0) # applica il 100% del carico in 1 passo
ops.algorithm('Linear')     # risolve il sistema una sola volta (solo per
strutture lineari)
ops.analysis('Static')      # tipo di analisi: statica (non dinamica)
ops.analyze(1)              # esegue l'analisi per 1 passo
```

*🔗 Per analisi non lineari: `algorithm('Newton')` itera fino a convergenza. Per analisi dinamica: `analysis('Transient')`.
Per carichi applicati gradualmente: `integrator('LoadControl', 0.1) + analyze(10)`.*

Prossimi argomenti

- Combinazioni di carico NTC2018 (coefficienti γ_G e γ_Q per SLU e SLE)
- Lettura di spostamenti di piano e drift tra piani
- Azioni sismiche — analisi modale e spettro di risposta
- Esportazione risultati in Excel o PDF
- Libreria anastruct per visualizzazioni avanzate

Fine dispensa — Buono studio!